REMM: Revisable Embodied Memory for Manipulation

Anonymous Authors

***Abstract*—Long-horizon manipulation requires robots to retain task-relevant history while revising conclusions no longer supported by physical evidence. Existing memory-augmented controllers largely focus on retaining and retrieving past observations, but updating a changed physical relation alone can leave dependent task conditions stale. We introduce REMM, Revisable Embodied Memory for Manipulation, a supervisory memory layer for a frozen G0.5 vision-language-action executor. REMM separates an append-only evidence history from a revisable task-state representation and records task-semantic dependencies among physical relations, derived scene state, and completed task conditions. When new evidence supersedes a relation, REMM retracts dependent conclusions, identifies the affected task obligation, and issues a bounded recovery instruction at the next stationary skill boundary without updating the executor. We evaluate REMM across 51 physical trials, including operator-assisted runs, spanning intervention recovery, ordered sequence recall, and hidden-object rearrangement. Relative to the strongest baseline in each task family, REMM improves mean recorded checkpoint progress from 57% to 91% for intervention recovery and from 43% to 70% for sequence recall, while completing all hidden-object trials. In matched offline diagnostics that include $\pi_{0.5}$-compatible high-level planner adaptations, REMM produces 8/12 fully grounded sequence decisions and the correct hidden-object choice and command on 30/30 clips; the strongest baseline reaches 5/12 and 13/30, respectively. Together, these evaluations examine how retained history and revisable task state can support continued manipulation without retraining the underlying robot policy. Paper website: https://remm-robot.github.io/**

I. Introduction

Long-horizon manipulation requires more than executing a sequence of independent skills. Each action changes the world in which subsequent decisions are made, and information needed later may no longer be directly observable: which object was handled first, what remains inside a container, or where an object was placed before becoming occluded. Such tasks therefore require decisions that depend on interaction history under partial observability [1], [2]. Yet retaining history alone is insufficient. A previously valid relation may cease to hold after a slip, human intervention, or environmental rearrangement. A robot must therefore preserve *what happened* while revising *what is currently supported by the available evidence*.

Generalist robot policies have substantially improved the execution of individual manipulation skills [3]–[7]. Recent models further extend language-conditioned control to long-horizon settings; in particular, G0.5 provides an autoregressive interface that interleaves reasoning and action generation within a single policy [7]. However, a capable executor does not by itself determine what should happen next when the relevant evidence is old, hidden, or contradicted by a subsequent physical intervention.

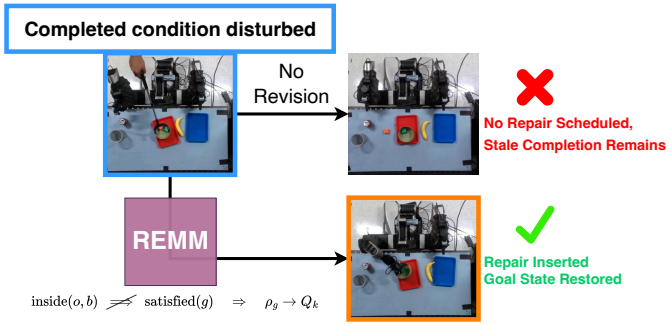


Fig. 1. **Revising memory after physical intervention.** A manipulation history can support either continued execution or recovery depending on whether later evidence preserves or contradicts an earlier task relation. REMM updates the supported task state and changes the subsequent robot action without retraining the frozen executor.

Recent work equips manipulation systems with temporal context through learned memory banks, sparse visual events, or retrieved keyframes [2], [8], [9]. Hierarchical systems instead place memory and reasoning above the motor policy: HiMe [10] maintains editable multimodal memory, while HyMeS [11] and BATON [12] steer reusable or frozen skills using external verification and transition-aware memory. These methods establish that memory can extend a policy beyond its immediate observation. They also sharpen a different question: after evidence changes, which earlier conclusions should change with it?

Closed-loop planners can incorporate success, scene, or human feedback into subsequent decisions [13]; structured world representations encode relational scene state [14]; and dynamic memories can add, delete, or modify stored relations as new evidence arrives [15]. Yet these capabilities do not explicitly track which physical relations support previously completed task conditions. If such a relation changes without revising its downstream consequences, the controller can retain an internally coherent but physically stale task state. We therefore consider an intervention-grounded process memory in which physical relations, derived scene state, and completed task conditions are connected through explicit support dependencies. When verified evidence changes an upstream relation, its affected consequences can be selectively withdrawn. These dependencies are supplied by task and physical semantics rather than learned through causal discovery; their use as recorded justifications is related to truth maintenance [16],

while remaining distinct from causal world models that learn or probabilistically infer manipulation outcomes [17], [18].

We introduce **REMM**, **R**evisable **E**mbodied **M**emory for **M**anipulation, a supervisory layer around a frozen G0.5 executor. REMM preserves an append-only evidence history while maintaining a separate projection of currently supported state. It records causal dependency links from physical relations to derived and completed task conditions; when verified evidence supersedes a parent relation, REMM invalidates affected descendants, identifies the broken task obligation, and schedules a bounded recovery instruction at the next safe skill boundary. This produces an inspectable path from physical intervention to memory revision and changed control without REMM-specific parameter training.

We evaluate REMM on real-robot tasks involving transport disruptions, delayed sequence recall, and hidden-object rearrangement. The evaluation separates *retention* from *revision*: sequence and hidden-object tasks test whether decision-relevant history persists through delay and occlusion, while transport interventions test whether previously valid task state is correctly withdrawn and repaired after the world changes. Beyond end-to-end task success, we measure change acceptance, dependency revision, recovery timing and destination, physical repair, and the observed memory-to-control trace. Across 51 physical trials, REMM improves mean checkpoint progress over the strongest baseline from 57% to 91% in intervention recovery and from 43% to 70% in sequence recall, while completing all hidden-object trials.

Our contributions are threefold:

- a formulation of *revisable embodied memory* that separates recorded execution history from currently supported task state and represents intervention-sensitive dependencies from physical relations to derived and completed task conditions;
- REMM, a supervisory system that propagates dependency invalidation and dispatches nominal or recovery instructions to a frozen G0.5 policy without REMM-specific parameter training; and
- a controlled real-robot evaluation that traces observed changes through memory revision to subsequent control decisions across transport, sequence-recall, and hidden-object manipulation tasks.

II. Related Work

A. Memory for Vision–Language–Action Control

Policy-integrated approaches make history part of action generation. MemoryVLA retrieves perceptual and cognitive tokens from a learned memory bank [2]; EventVLA learns when to preserve sparse visual evidence [8]; and Chameleon and ChainVLA propagate compact event or execution state across action queries [19], [20]. These methods learn representations for retention and retrieval inside the policy. RoboMME shows that their benefit depends strongly on whether a task requires temporal, spatial, object, or procedural memory [1]. REMM uses a different boundary: it leaves G0.5 frozen, does not learn a memory-conditioned action model, and studies explicit revision of task state between bounded skills.

Hierarchical approaches place memory above the executor. MemER trains a high-level visual policy to select keyframes and predict subtasks for a separately trained $\pi_{0.5}$ [9]. HiMe divides control among an Executor, a working-memory Sentry, and a Planner that retrieves and edits multimodal memory [10]. HyMeS uses executable memory-management heuristics and multimodal completion checks to steer a Markovian VLA [11], while BATON uses a frozen VLA with subtask exploration and transition-aware memory [12]. These systems are the closest architectural context for REMM. Our question is narrower than general memory management: whether storing explicit parents for derived and completed task conditions allows a verified intervention to revise exactly the downstream obligations that no longer hold.

B. Closed-Loop State Revision and Recovery

Robotic planners have long used feedback to reconsider high-level action. Inner Monologue conditions an LLM planner on success detection, scene description, and human feedback [13]; $\mathcal{P}^3$ schedules emerging tasks using priorities, dependencies, interruption, and resumption [21]; and PhyAgentOS couples persistent session state with verification and replanning [22]. These works establish closed-loop supervisory control, but their task scheduling or model-mediated replanning does not by itself expose which physical fact sustained a completed condition. REMM connects these levels explicitly and preserves the active atomic skill while inserting, deferring, or retargeting a recovery at a skill boundary.

Structured memories provide the complementary world-state representation. ReMEmbR indexes spatially and temporally grounded experience for navigation [23]; ConceptGraphs builds open-vocabulary object–relation graphs for planning [14]; and RoboMemory revises relevant neighborhoods of temporal, spatial, semantic, and episodic memory [15]. REMM does not claim the first relational memory or editable knowledge graph. It adds an append-only evidence ledger, an active projection, and typed links from physical relations to derived scene and task conditions.

Those links draw on two related traditions. Truth-maintenance systems retain justifications and withdraw conclusions when their support becomes inactive [16]. Causal manipulation work instead uses controlled interventions or causal Bayesian models to reason about action effects [17], [18]. REMM does not discover a causal graph or predict general dynamics. It instantiates task-semantic causal dependencies whose operational meaning is intervention-sensitive: changing a required parent relation changes the validity of its descendant task condition and, through the controller, the selected continuation. The experiments therefore test both the stored revision and its downstream control effect rather than treating an edited memory record as sufficient evidence of recovery.

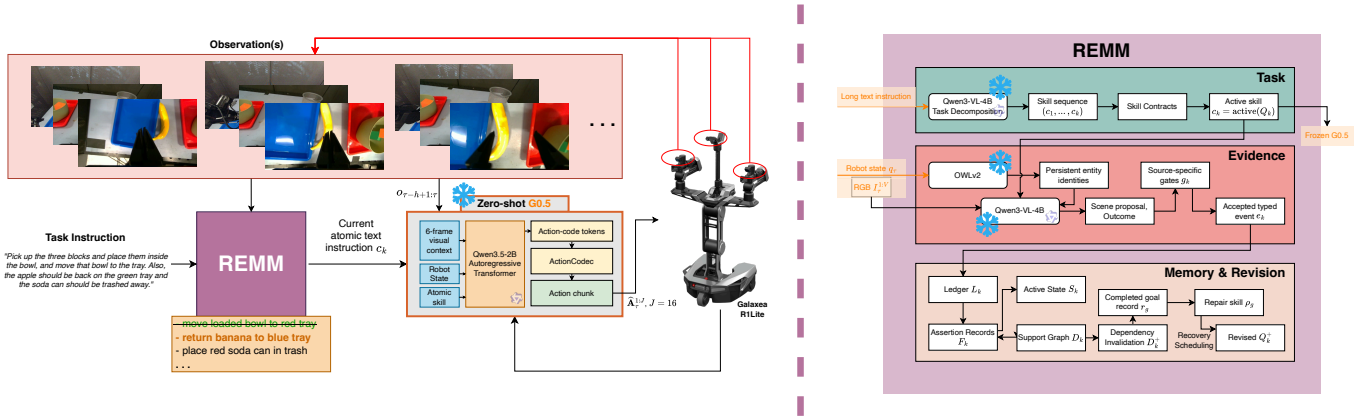


Fig. 2. **System and memory architecture.** Left: REMM converts a long-horizon instruction and multi-view observations into the current atomic instruction for a frozen G0.5 executor, which predicts action-code tokens and action chunks for the Galaxea R1 Lite; new observations and robot state close the loop after execution. Right: a task decomposer and skill contracts define the active skill, frozen perception and visual verification produce accepted typed events, and an append-only ledger maintains an active-state projection and support graph. When new evidence supersedes a supporting fact, dependency invalidation withdraws affected goal records before REMM schedules a repair skill or issues the revised next query. Snowflakes denote frozen models.

III. Method

A. Problem Setting and System Overview

The robot receives episode instruction ℓ and observations $o_\tau = (I_\tau^{1:V}, q_\tau)$, comprising synchronized images from V views and proprioception. At each policy query τ, the frozen G0.5 executor consumes its latest h observations, REMM-selected atomic instruction c_k, and R1 Lite embodiment identifier e_{R1}:

$$\left(y_\tau, \widehat{\mathbf{A}}_\tau^{1:J}\right) = \pi_{\mathrm{G0.5}}(o_{\tau-h+1:\tau}, c_k, e_{\mathrm{R1}}; \theta_0), \quad \theta_0 \text{ fixed.} \quad (1)$$

The outputs are optional grounded reasoning tokens y_τ and a continuous action chunk $\widehat{\mathbf{A}}_\tau^{1:J}$. In our deployment, the client executes $J = 16$ actions at 15 Hz. REMM holds c_k fixed during the skill and selects a new instruction only at a verified boundary; the executor parameters θ_0 remain frozen.

At boundary k, REMM maintains $M_k = (L_k, F_k, D_k, Q_k)$: the append-only event ledger L_k, assertion records F_k, directed support graph D_k, and ordered skill queue Q_k. The currently dispatched instruction is $c_k = \text{active}(Q_k)$.

Separate from the G0.5 executor, REMM uses frozen Qwen3-VL-4B-Instruct [24] for constrained task decomposition and visual verification, and OWLv2-base-patch16-ensemble [25] for open-vocabulary 2D object proposals. Deterministic validators and source-specific evidence gates convert these outputs into accepted typed events; T2 and T3 use task-specific memory-query compilers rather than open-ended Qwen planning.

B. Revisable Memory State

Let $\varphi_i \equiv [p_i(\mathbf{x}_i) = v_i]$ be the grounded proposition in assertion record r_i, where p_i is a predicate, $\mathbf{x}_i$ contains its entity arguments, and $v_i \in \{\text{true}, \text{false}, \text{unresolved}\}$. Each record also stores confidence γ_i, evidence references $\mathcal{E}_i$, direct parents $\mathcal{P}_i$, and status $z_i \in \{\text{active}, \text{superseded}, \text{invalidated}\}$:

$$\begin{aligned} r_i &= (\varphi_i, \gamma_i, \mathcal{E}_i, \mathcal{P}_i, z_i), \\ S_k &= \{\varphi_i \mid r_i \in F_k,\ z_i = \text{active}\}. \end{aligned} \quad (2)$$

Each $r_j \in \mathcal{P}_i$ creates edge $r_j \to r_i$ in D_k. The record set F_k preserves active and inactive assertions, while S_k projects only the propositions REMM currently treats as supported. These two views answer different questions. The ledger records that an object was placed in a bowl; the projection answers whether its containment is still supported now. A later intervention can change the second answer without erasing the first.

Each derived record has one conjunctive support set: every parent is required. The implementation does not maintain competing alternative justifications. Losing any required parent therefore withdraws the derived record until new accepted evidence supports it again. An invalidated record is no longer justified by the stored evidence; it is not automatically a new observation that the proposition is false.

A candidate event e_k contains a proposed state change and its provenance. Binary gate g_s applies the acceptance rule for source $s = \text{src}(e_k)$; only accepted evidence enters the ledger:

$$L_{k+1} = \begin{cases} L_k \parallel e_k, & g_{\text{src}(e_k)}(e_k) = 1, \\ L_k, & \text{otherwise}, \end{cases} \quad (3)$$

where $\parallel$ denotes append. Visual relations require grounded identities, an allowed relation, noncontradictory evidence, confidence of at least 0.8, and two distinct agreeing observations. Operator-verified outcomes and geometric contradictions use separate gates; a missing detection alone never retracts a fact. Confidence is a source-specific acceptance signal, not a calibrated probability of correctness.

This boundary keeps a model proposal separate from an accepted change. An object disappearing from one camera view may reflect occlusion rather than motion, and a policy's completion statement may precede a failed release. Neither is sufficient by itself to revise the task state. Accepted events

retain their entity and frame references so that the controller's later decision can be traced back to the evidence that made it valid.

C. Dependency Revision and Recovery

REMM separates direct physical assertions from their consequences. Let o be an object, c a container or support, l a coarse task region, and $R \in \{inside, on\}$. If active records place o relative to c and c at l, REMM derives the supported location of o:

$$\begin{aligned} r_s &: R(o,c) = \top, \qquad R \in \{inside, on\}, \\ r_l &: at(c,l) = \top, \\ r_s, r_l &\Longrightarrow r_d : at(o,l) = \top, \\ \mathcal{P}_d &= \{r_s, r_l\}, \quad \gamma_d = \min(\gamma_s, \gamma_l). \end{aligned} \tag{4}$$

Here l is a symbolic workspace region. The derived record stores both parents and inherits the weaker confidence, creating an explicit dependency that can later be withdrawn [26], [27].

For atomic placement goal g, $\mathrm{verify}_k(g) = 1$ denotes an accepted physical postcondition at boundary k. REMM represents completion as a derived record supported by the verified physical relation:

$$\begin{aligned} &\mathrm{verify}_k(g) = 1, \quad R \in \{inside, on, at\}, \\ &r_s : R(o,d) = \top \\ &\quad \Longrightarrow r_g : satisfied(g) = \top, \\ &\mathcal{P}_g \supseteq \{r_s\}. \end{aligned} \tag{5}$$

Goal completion therefore remains linked to the observed physical condition that justified it. This follows the general principle of semantic execution monitoring—checking expected action effects against observations [28]—while REMM additionally records the supporting fact.

Edges in D_k encode declared support relations, related to causal-link planning and task decomposition [29]; REMM supplies these links from task semantics rather than learning them from trajectories. Let D_k^+ denote reachability through one or more support edges. When record r_i becomes inactive, REMM invalidates every active descendant r_j:

$$\begin{aligned} &z_i \neq \text{active}, \quad (r_i, r_j) \in D_k^+, \\ &z_j = \text{active} \implies z_j \leftarrow \text{invalidated}. \end{aligned} \tag{6}$$

As in truth maintenance, this operation preserves the evidence history while withdrawing only conclusions supported by the changed relation [16]. If invalidation reaches completed-goal record r_g, REMM creates or updates repair skill ρ_g. Let $E_g = \mathrm{ent}(\rho_g)$ and $E_k = \mathrm{ent}(c_k)$ contain the persistent entities involved in the repair and active instruction. Their overlap determines the queue update:

$$Q_k^+ = \begin{cases} \mathrm{insert}_{\mathrm{next}}(Q_k, \rho_g), & E_g \cap E_k \neq \varnothing, \\ \mathrm{append}(Q_k, \rho_g), & \text{otherwise}. \end{cases} \tag{7}$$

Here Q_k^+ is the post-revision queue. The operator $\mathrm{insert}_{\mathrm{next}}$ places the repair immediately after the active skill, whereas append defers it behind the remaining work. Both operations deduplicate equivalent repairs and may retarget a pending repair using newer evidence. The active c_k is unchanged; the controller dispatches from Q_k^+ only at a stationary skill boundary.

For example, suppose a verified placement supports both $inside(o, b)$ and the completed goal of transporting o with bowl b. If later evidence places o outside the bowl, the earlier containment record is superseded, its inferred destination and completion lose support, and the transport obligation becomes pending again. The bowl's independently verified location and unrelated completed placements remain active. The repair is compiled from the current supported state, rather than replaying a stale instruction. If the required destination changes before dispatch, the pending repair can change with it.

D. From Retained History to an Atomic Instruction

Revision is only one use of the same memory boundary. In T2, the demonstrated cue order remains historical information even after the cues are covered. The controller must then bind a remembered cue to an object visible in the current scene and resolve the placement destination. Retaining the color sequence alone does not supply either binding. The task-specific compiler uses these records to form the next bounded placement instruction; the executor receives that instruction and its native observations, not the full event ledger.

T3 uses a different retained relation: the target object was covered by a particular cup. The selected cup must follow that persistent identity through rearrangement rather than being re-identified only by its initial left, middle, or right position. The offline implementation uses pixel-initialized MIL tracking for this purpose [30]; a task compiler converts the tracked cover's final position into the lift instruction. This specialized path is distinct from T1 dependency revision and from open-ended VLM planning. We therefore evaluate retention, grounding, and repair separately rather than attributing every successful memory query to the support graph.

IV. Experiments

We evaluate REMM across 51 physical robot trials and 42 matched generated-video clips. Our experiments address three questions:

1) **Revision and recovery:** When new physical evidence contradicts a previously satisfied task condition, can REMM invalidate the dependent state and schedule the correct repair?
2) **Retention and grounding:** Can REMM preserve ordered and object-identity information through delay, occlusion, and rearrangement, then ground that history into the correct next instruction?
3) **Control effect and cost:** With the G0.5 executor held fixed, how do these memory decisions affect task progress, end-to-end success, failure modes, and system cost relative to no-memory and hierarchical-memory baselines?

TABLE I

EVALUATION COVERAGE BY TASK. CHECKS MARK MECHANISMS TESTED BY AT LEAST ONE PREDECLARED CONDITION. T2 COMBINES NOMINAL ORDERED RECALL WITH DISTURBED REPEATS; ONLY THE DISTURBED REPEATS TEST REVISION AND RECOVERY.

		Memory demands			Revision demands			
Task	Condition	Retention	Order	Identity	Revision	Invalidation	Recovery	Retargeting
T1a	Queued disruption	✓	×	×	✓	✓	✓	×
T1b	Destination shift after pouring	✓	×	×	✓	✓	✓	✓
T2	Ordered recall + disturbed repeats	✓	✓	×	✓	✓	✓	×
T3	Hidden-object shuffle	✓	×	✓	×	×	×	×

A. Task Families

We organize the study by memory demand, rather than by atomic motor skill, as summarized in Fig. 3.

Transport and recovery tests deferring a correction while unrelated work remains queued (T1a) and retargeting recovery after a destination change (T1b).

Sequence recall (T2) tests recording a hidden demonstration order, grounding each historical cue to a current object, and compiling an ordered multi-skill queue; predeclared disturbed repeats additionally test whether completed placements are revised and repaired without corrupting the remaining order.

Hidden-object search (T3) instead tests whether one object–cover binding remains attached to the same cover through a shuffle.

B. Dependency Mechanism Controls

To distinguish stored dependency invalidation from observing the same physical change, we define three state-update strategies. Let $\mathcal{V}_k$ contain completed-goal records newly invalidated at boundary k, let $h_k(r) = 1$ when support record r agrees with the latest direct facts, and let $b_k^\sigma(g)$ indicate that strategy σ declares goal g broken:

$$b_k^\sigma(g) = \begin{cases} \mathbb{I}[r_g \in \mathcal{V}_k], & \sigma = \text{explicit}, \\ \mathbb{I}[\exists r \in \mathcal{P}_g : h_k(r) = 0], & \sigma = \text{recompute}, \\ 0, & \sigma = \text{direct}. \end{cases} \tag{8}$$

The explicit strategy reads REMM's stored invalidation, recomputation checks the saved postconditions on demand, and direct update changes only the physical relation. All three receive the same accepted evidence; this is a software-level mechanism control, not an external baseline. The full-system physical comparisons below do not isolate an advantage of stored propagation over recomputing the affected goals.

C. Evaluation Metrics

For trial e, let $\mathcal{G}_e = (g_{e,1}, \ldots, g_{e,K_e})$ be evaluator-defined task checkpoints fixed before execution, and let $y_{e,m} \in \{0, 1\}$ indicate whether checkpoint m is independently satisfied. We report both partial task progress P_e and strict end-to-end success S_e:

$$\begin{aligned} P_e &= \frac{1}{K_e} \sum_{m=1}^{K_e} y_{e,m}, \\ S_e &= \prod_{m=1}^{K_e} y_{e,m}. \end{aligned} \tag{9}$$

Checkpoints are fixed before execution and shared across methods; they never use a controller's self-generated plan. Equation 9 defines the intended checkpoint measure. The plotted trial export uses recorded progress when available and completion-label or T3 decision-label fallbacks when progress is absent. Full, partial, and failed outcomes retain their recorded labels rather than being reconstructed by thresholding progress. For disturbed trials, accepted change, invalidated task state, recovery placement in the queue, and physical repair are distinct events; an observed memory update is not itself physical success.

$$\begin{aligned} B_w &= \sum_{i=1}^{N_w} |\text{serialize}(W_i)|, \\ B_r &= \sum_{j=1}^{N_r} |\text{serialize}(R_j)|, \\ \bar{n}_w &= N_w/N_q, \qquad \bar{n}_r = N_r/N_q, \\ \bar{b}_w &= B_w/N_q, \qquad \bar{b}_r = B_r/N_q. \end{aligned} \tag{10}$$

Here W_i and R_j are method-native logical write and retrieval payloads, and N_q counts fresh G0.5 action-chunk queries. Current observations, audit recordings, and model weights are reported separately rather than counted as memory traffic.

V. RESULTS

We report three complementary views: physical outcome decomposition, task-level decision and cost profiles, and matched offline diagnostics of high-level memory readout. The physical export uses saved trial-ledger outcomes and operator annotations. Each rate uses the available eligible labels for its own denominator, and blank labels are excluded from that rate rather than silently treated as failures. Appendix B gives the task counts and assistance accounting.

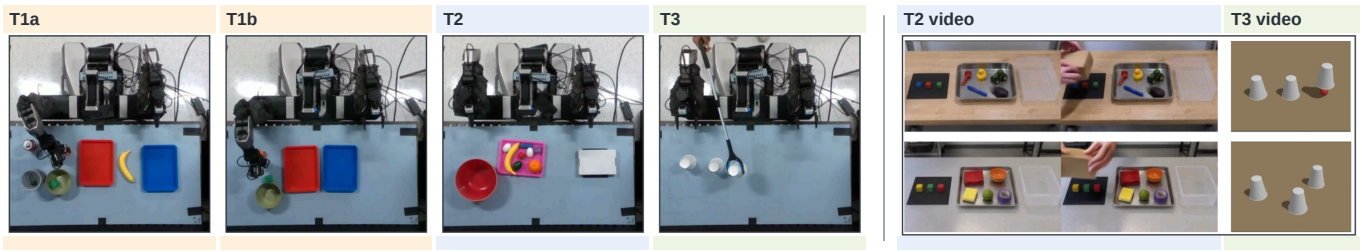


Fig. 3. **Physical and offline task families.** The enlarged panels show the T1a/T1b recovery setups, T2 ordered manipulation, and T3 hidden-object manipulation alongside representative matched T2 and T3 video diagnostics.

A. End-to-End Physical Outcomes

Figure 4 separates full task success from partial completion and failure. REMM is the only system with a full completion in every task family: 10 of 17 trials overall, compared with 2 of 16 for HiMe and 0 of 18 for vanilla G0.5. None of the 17 REMM trials is classified as a complete failure, although seven remain partial. These pooled counts are descriptive rather than a single comparable success rate because the task mix and denominators differ; the task-separated checkpoints below provide the more informative comparison.

These outcomes include operator-assisted runs. Two of REMM's full completions have recorded task assistance, so the pooled counts are not unassisted-autonomy success rates. HiMe denotes our controlled adaptation around the shared G0.5 executor, rather than an official reproduction of the original model stack.

B. Task-Level System Profiles

Figure 4 reports checkpoint progress, decision accuracy, non-failure completion, execution-time efficiency, and G0.5-query efficiency separately for T1, T2, and T3; T1 pools T1a and T1b. The first three measures are observed percentages. For cost x, method m on task t receives efficiency $100 \min_{m'} \widetilde{x}_{m',t} / \widetilde{x}_{m,t}$, where $\widetilde{x}_{m,t}$ is the median observed cost. This normalization preserves proportional cost differences while assigning 100 to the most efficient observed method per task.

The checkpoint profiles explain what the pooled outcomes hide. On T1, REMM raises mean progress from 57% for the strongest baseline to 91%; on T2, it raises progress from 43% to 70%; and on T3 it completes all four trials while neither baseline records a full completion. This gain carries a visible systems cost: REMM is not the most time- or query-efficient method on every task because it performs additional perception and verification and often continues runs that a baseline terminates early. We therefore report efficiency beside success rather than folding both into one score.

G0.5-query efficiency counts only fresh action-chunk queries, not planner, sentry, or verifier VLM calls. It is therefore not a measure of total inference cost. In T2, the baseline time and query medians each use only two of five trials. Missing records, assistance, and early stopping limit what can be concluded from these relative efficiencies; they do not establish lower cost at equal task performance.

C. Offline Generated-Video Diagnostics

The generated-video comparison uses the same 12 T2 clips and 30 T3 clips for every method; representative frames appear in Fig. 6. T2 uses endpoint-conditioned LTX-Video 0.9.8 13B Distilled [31], [32], with OpenAI-generated endpoint images. T3 uses deterministic VET-Bench shell-game renders [33], rather than generative-model video. The scored snapshot stores human approval verdicts but not reviewer-count or adjudication records; a blinded second review remains needed for a fully documented benchmark release.

T2 measures cue-order recall, exact three-item sequence, first-object and destination grounding, and a grounded first command; T3 requires one left, middle, or right cup choice and its corresponding lift command. The command scorer requires the correct action, object or color, and destination while allowing a fixed set of transparent noun equivalences. The no-memory condition uses final-frame Qwen3-VL-4B-Instruct, not the G0.5 motor policy. Adapted HiMe adds episodic and procedural memory. The MemER-style [9] and RoboMME-style [1] conditions use frozen Qwen with visual-keyframe and symbolic-subgoal memory interfaces, respectively. They emit canonical language subtasks compatible with a downstream $\pi_{0.5}$ executor [6]; they are declared adaptations rather than official reproductions, and the passive-video diagnostic does not run the $\pi_{0.5}$ motor policy. No manifest-derived inventory, destination, expected command, or target answer is supplied to any method. Appendix C describes what each output is compared against.

The T2 columns in Table II and the three panels in Fig. 5 separate remembering a sequence from turning that memory into an actionable instruction. MemER-style retains all 12 sequences but grounds neither the first object nor the command; RoboMME-style reaches 10/12 exact sequences, 6/12 first objects, and 6/12 commands. HiMe and REMM both achieve cue LCS 35/36 and exact sequence 11/12, yet HiMe grounds the first object and command on only 1 of 12 clips, versus 11 of 12 and 8 of 12 for REMM.

The five-metric T2 partial score is 86.1% for REMM, compared with 73.9% for RoboMME-style, 55.0% for MemER-style, 54.4% for HiMe, and 38.9% for no-memory. This descriptive average normalizes cue LCS by 36 and each remaining component by 12 before assigning equal weights. It is not the percentage of successful clips, and its two

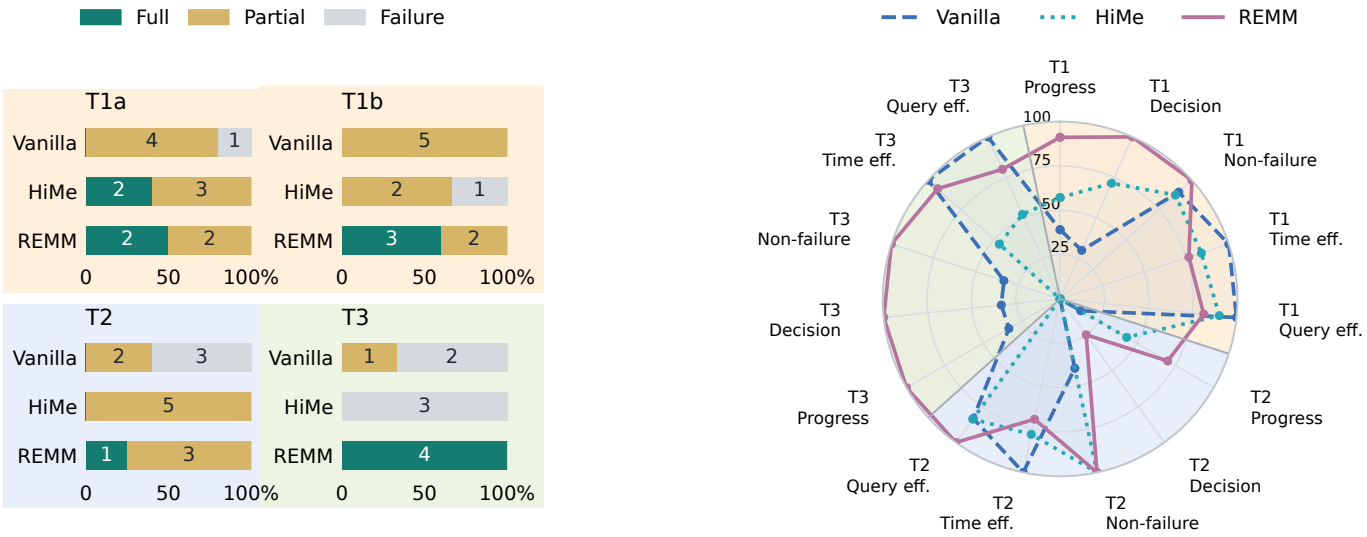


Fig. 4. **Physical outcomes and system profiles.** Left: full, partial, and failed outcomes by task, with raw trial counts. Right: progress, decision accuracy, non-failure completion, time efficiency, and G0.5-query efficiency in task-colored sectors. T1 pools T1a/T1b; higher is better. Assisted runs are included, and HiMe denotes our adaptation. Non-failure includes partial trials; efficiency is $100\times$ best task-specific median cost / method median.

TABLE II

MATCHED OFFLINE VIDEO DIAGNOSTIC. COMPONENTS ARE RAW COUNTS OVER 12 T2 AND 30 T3 CLIPS. T2 PARTIAL AVERAGES FIVE NORMALIZED METRICS; FULL T2 REQUIRES ALL FOUR DECISION FIELDS ON THE SAME CLIP. THE $\pi_{0.5}$-COMPATIBLE ROWS EVALUATE HIGH-LEVEL PLANNER ADAPTATIONS, NOT MOTOR EXECUTION [6].

	T2: ordered recall and grounding							T3
Method	T2 cue LCS /36	Exact seq. /12	First object /12	Destination /12	Grounded command /12	Partial T2	Full T2 success	T3 hidden target
No-memory VLM [24]	19	1	3	11	2	38.9%	0.0% (0/12)	36.7% (11/30)
HiMe [10]	35	11	1	8	1	54.4%	0.0% (0/12)	33.3% (10/30)
MemER-style [9]	36	12	0	9	0	55.0%	0.0% (0/12)	43.3% (13/30)
RoboMME-style [1]	34	10	6	11	6	73.9%	41.7% (5/12)	36.7% (11/30)
REMM	35	11	11	10	8	**86.1%**	**66.7% (8/12)**	**100.0% (30/30)**

recall components are related measures rather than independent outcomes. On the stricter per-clip conjunction—correct sequence, first object, destination, and command—REMM achieves 8/12 fully grounded T2 decisions, compared with 5/12 for RoboMME-style and 0/12 for MemER-style, HiMe, and no-memory. MemER-style's one-clip advantage in pure sequence retention and RoboMME-style's one-clip advantage in destination recognition therefore do not translate into a higher memory-to-action success rate; destination is visible in the final scene and is not itself the principal memory bottleneck.

In T3, the no-memory VLM always chooses the middle cup and therefore matches the fixed-set majority at 11/30; HiMe reaches 10/30, MemER-style 13/30, and RoboMME-style 11/30. REMM's persistent target–cover track is correct on all 30 deterministic clips (Wilson 95% interval: 88.6–100%). This result is specific to the generated set and does not establish perfect tracking under real occlusion or appearance shift. These are high-level offline diagnostics rather than physical manipulation outcomes: passive video never executes G0.5, and observation selection and compute follow each method's architecture. In particular, REMM uses eight sampled views with Qwen and event memory for T2, and a pixel-initialized MIL tracker with a task compiler for T3. These unequal observation and compute paths limit component-level conclusions.

VI. DISCUSSION

A. *Interpreting the Results*

The results support separating retention, revision, and grounding rather than treating embodied memory as a single capability. Nominal T2 and T3 test whether task-relevant information remains available after delay, covering, or rearrangement. T1 and the disturbed T2 repeats additionally require a previously supported task condition to become inactive. REMM is designed for this second case: accepted evidence is connected to the completed condition it supported, allowing a change to withdraw that condition and create a bounded repair instead of merely adding another observation to an undifferentiated history.

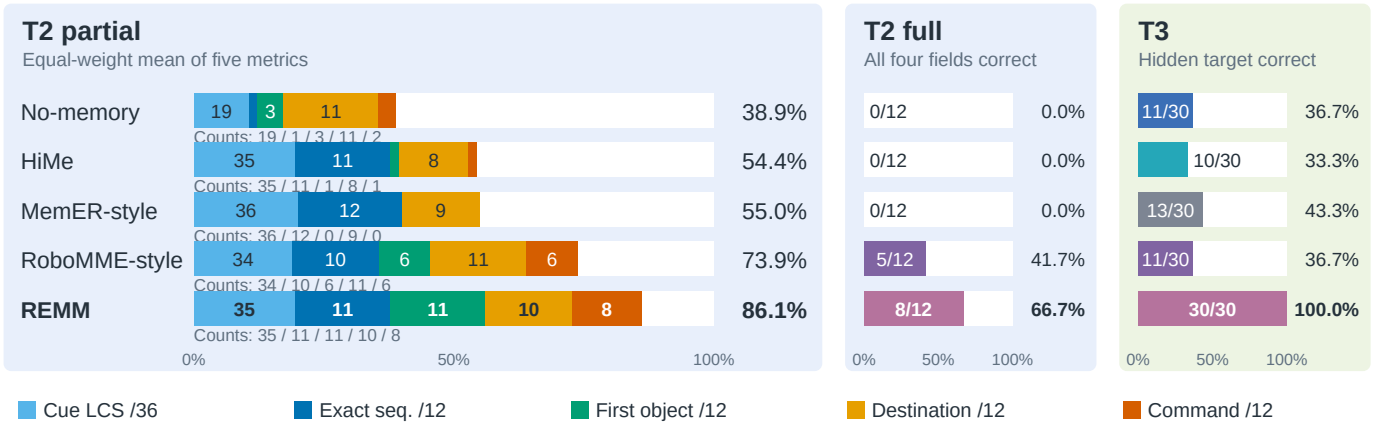


Fig. 5. **Offline video results.** T2 partial averages five normalized accuracies with equal weights; segment counts follow legend order. Full T2 requires correct sequence, first object, destination, and command on the same clip (/12). T3 scores hidden-target choice (/30). No motor policy is executed. Exact counts are in Table II.

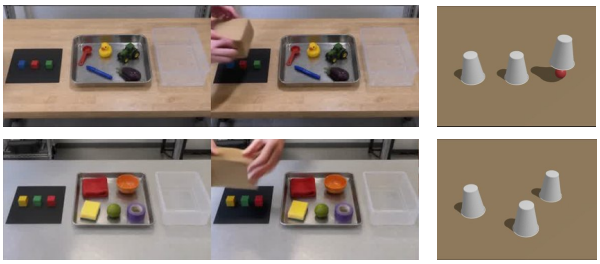


Fig. 6. **Generated-video diagnostics.** Left: T2 requires retaining a demonstrated color order after the cues are covered. Right: T3 requires preserving the hidden object's cover identity through a rearrangement. These diagnostics score high-level memory readout and command generation without executing the robot.

The offline T2 result clarifies where structured memory matters. MemER-style slightly exceeds REMM on pure sequence storage, and RoboMME-style slightly exceeds it on visible destination recognition, but neither exceeds REMM on the fully grounded decision. HiMe recalls the demonstrated order nearly as well as REMM but rarely grounds that order to the correct first object or command. REMM's advantage therefore lies in persistent entity binding and compilation of retained state into an atomic instruction, rather than universal dominance of every diagnostic column. The comparison remains system-level: each method uses its architecture-specific observation-selection and inference path, and the $\pi_{0.5}$-compatible conditions are evaluated only above the motor-policy boundary, so the result does not attribute the difference to one isolated component, memory size, or low-level executor.

Holding G0.5 fixed removes policy training as a confound and makes the selected high-level instruction auditable, but it does not make every end-to-end failure a memory failure. The methods can differ in when they query observations, when they stop, and which instruction they send; the common executor can still fail to realize a correct instruction. We therefore interpret decision correctness, checkpoint progress, and physical success together. Likewise, the dependency controls isolate state-update logic in software, while the physical comparison measures complete systems; the physical gains should not be attributed to graph propagation alone.

B. Scope and Limitations

REMM's dependency graph is supplied by the task compiler and a fixed predicate ontology. The recorded links have an intervention-sensitive causal interpretation within those declared task semantics: changing a required parent relation changes the validity of its dependent task condition. REMM does not discover causal structure from trajectories, predict arbitrary physical effects, or automatically compile unconstrained language into new dependency rules.

The live state estimate also remains bounded by perception and execution. It uses conservative 2D detections, persistent identifiers, frozen VLM judgments, and source-specific evidence gates rather than calibrated metric 3D state. Missing detections do not retract facts, which reduces false revision but can delay necessary updates. Corrections are dispatched only at verified stationary skill boundaries; REMM does not interrupt an unsafe action chunk in progress or guarantee recovery when identity is lost under complete occlusion.

Finally, the physical study uses one robot, a small tabletop object set, controlled interventions, and limited per-condition samples. HiMe is evaluated as a controlled adaptation around the shared executor, so the comparison characterizes these deployed systems rather than every configuration of the original method. The generated clips provide matched diagnostic questions but remain deterministic and use architecture-specific compute paths. The MemER-style and RoboMME-style rows are likewise controlled, $\pi_{0.5}$-compatible high-level adaptations, not official reproductions or measurements of $\pi_{0.5}$ manipulation success. Broader evaluation should run the emitted subtasks through a matched low-level policy, test automatically constructed but auditable dependencies, calibrate 3D identity tracking, and study safe revision during execution

across additional robots, policies, and task families. To support reproducibility, we will release our implementation, evaluation protocols, controller configurations, structured memory traces, and full trial videos through the project website.

VII. Conclusion

Long-horizon manipulation requires a robot to retain useful history while withdrawing conclusions that later physical evidence no longer supports. REMM addresses this problem with an append-only evidence ledger, a revisable task-state projection, and explicit task-semantic dependencies that connect physical relations to completed conditions and bounded recovery instructions. Across 51 physical trials, REMM improves mean checkpoint progress on intervention-recovery tasks from 57% for the strongest baseline to 91%. In matched video diagnostics, including $\pi_{0.5}$-compatible high-level planner adaptations, REMM achieves 8/12 fully grounded T2 decisions versus 5/12 or fewer for every baseline, showing that retaining a sequence is insufficient unless it can be grounded into the next action. These results support revisable memory as a practical supervisory interface for a frozen VLA, while leaving open the harder problems of learning task dependencies, maintaining identity under unconstrained perception, and revising control safely within an active skill.

References

[1] Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai, "RoboMME: Benchmarking and understanding memory for robotic generalist policies," *arXiv preprint arXiv:2603.04639*, 2026. [Online]. Available: https://arxiv.org/abs/2603.04639

[2] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang, "Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation," *arXiv preprint arXiv:2508.19236*, 2025.

[3] A. Brohan *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.

[4] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees *et al.*, "Octo: An open-source generalist robot policy," in *Robotics: Science and Systems*, 2024.

[5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[6] Physical Intelligence *et al.*, "$\pi_{0.5}$: A vision–language–action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.

[7] Y. Liu, Z. Dong, B. Ye, T. Yuan, T. Jiang, A. Yang, S. Cao, H. Liu, Y. Sun, Z. Guo, X. Liu, D. Ke, C. Pan, C. Wu, T. Cheng, X. Ren, X. Zhang, J. Cui, Z. Zhao, H. Zhang, K. Xu, H. Yang, B. Zhang, J. Niu, S. Zhu, S. Zhang, and H. Zhao, "G0.5: One autoregressive stream for robot reasoning and action," 2026. [Online]. Available: https://arxiv.org/abs/2608.11739

[8] G. Yang, Z. Tu, Y. Yang, S. Mao, J. Dong, T. Chen, J. Peng, J. Xiong, J. Cao, J. Dai, W. Zhou, Y. Mu, and T. Wang, "EventVLA: Event-driven visual evidence memory for long-horizon vision-language-action policies," *arXiv preprint arXiv:2606.20092*, 2026.

[9] A. Sridhar, J. Pan, S. Sharma, and C. Finn, "MemER: Scaling up memory for robot control via experience retrieval," *arXiv preprint arXiv:2510.20328*, 2025.

[10] L. Ji, S. Wang, P. Qian, X. Yu, Y. Tian, Z. Fei, J. Gong, and X. Qiu, "HiMe: Hierarchical embodied memory for long-horizon vision-language-action control," *arXiv preprint arXiv:2607.03449*, 2026.

[11] Y. Zhao, Z. Ni, H. Chen, R. Zhang, and Q. Zhu, "Skills in weights, memory in code: Hybrid learning for memory-dependent robot manipulation," *arXiv preprint arXiv:2608.09410*, 2026.

[12] B. Xu, Y. Shang, and E. Ferrara, "Don't drop the BATON: Long-horizon robot manipulation via agentic subtask exploration and transition-aware memory," *arXiv preprint arXiv:2608.16889*, 2026.

[13] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng *et al.*, "Inner monologue: Embodied reasoning through planning with language models," *arXiv preprint arXiv:2207.05608*, 2022.

[14] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, C. Gan, C. M. de Melo, J. B. Tenenbaum, A. Torralba, F. Shkurti, and L. Paull, "ConceptGraphs: Open-vocabulary 3d scene graphs for perception and planning," in *2024 IEEE International Conference on Robotics and Automation*, 2024, pp. 5021–5028.

[15] M. Lei, H. Cai, Y. Yang, Y. Wu, J. Ren, Z. Cui, L. Tan, J. Hong, G. Hu, S. Zhu, S. Jiang, G. Wang, J. Tan, Z. Wan, Z. Li, Z. Li, S. Cui, Y. Zhao, and Y. Han, "RoboMemory: A brain-inspired multi-memory agentic framework for interactive environmental learning in physical embodied systems," 2026. [Online]. Available: https://arxiv.org/abs/2508.01415

[16] J. Doyle, "A truth maintenance system," *Artificial Intelligence*, vol. 12, no. 3, pp. 231–272, 1979.

[17] O. Ahmed, F. Träuble, A. Goyal, A. Neitz, Y. Bengio, B. Schölkopf, M. Wüthrich, and S. Bauer, "CausalWorld: A robotic manipulation benchmark for causal structure and transfer learning," in *International Conference on Learning Representations*, 2021.

[18] R. Cannizzaro, M. Groom, J. Routley, R. O. Ness, and L. Kunze, "COBRA-PPM: A causal bayesian reasoning architecture using probabilistic programming for robot manipulation under uncertainty," in *2025 European Conference on Mobile Robots*, 2025, pp. 1–8.

[19] X. Guo, C. Jiang, H. B. Kim, Y. Han, Y. Sun, Y. Xiao, and J. Yang, "Chameleon: Control-indexed prospective memory for visuomotor manipulation," *arXiv preprint arXiv:2603.24576*, 2026.

[20] Y. Huang, W. Bu, Z. Xiong, J. Wu, F. Huang, J. Jiang, and Z. Wang, "ChainVLA: Chaining vision-language-action queries through a unified execution state for long-horizon manipulation," *arXiv preprint arXiv:2608.02326*, 2026.

[21] S. Zhou, X. Wang, J. Zhang, R. Tian, J. Huang, R. Xu, G. Chen, and F. Zheng, "$\mathcal{P}^3$: Toward versatile embodied agents," 2026. [Online]. Available: https://arxiv.org/abs/2508.07033

[22] Y. Liu, W. Chen, X. Song, T. Pu, S. Mo, Y. Bai, Z. Chen, Q. Sun, L. Zhong, Y. Shen, and L. Lin, "PhyAgentOS: A self-evolving operating system for embodied agents with decoupled cognitive planning and physical execution," *arXiv preprint arXiv:2607.16636*, 2026.

[23] A. Anwar, J. Welsh, J. Biswas, S. Pouya, and Y. Chang, "ReMEmbR: Building and reasoning over long-horizon spatio-temporal memory for robot navigation," *arXiv preprint arXiv:2409.13682*, 2024.

[24] S. Bai, Y. Cai, R. Chen *et al.*, "Qwen3-VL technical report," *arXiv preprint arXiv:2511.21631*, 2025. [Online]. Available: https://arxiv.org/abs/2511.21631

[25] M. Minderer, A. Gritsenko, and N. Houlsby, "Scaling open-vocabulary object detection," in *Advances in Neural Information Processing Systems*, 2023.

[26] W. Liang, Y. Zhao, Y. Zhu, and S.-C. Zhu, "What is where: Inferring containment relations from videos," in *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, 2016, pp. 3418–3424.

[27] M. Tenorth and M. Beetz, "Representations for robot knowledge in the KnowRob framework," *Artificial Intelligence*, vol. 247, pp. 151–169, 2017.

[28] A. Bouguerra, L. Karlsson, and A. Saffiotti, "Monitoring the execution of robot plans using semantic knowledge," *Robotics and Autonomous Systems*, vol. 56, no. 11, pp. 942–954, 2008.

[29] R. M. Young, M. E. Pollack, and J. D. Moore, "Decomposition and causality in partial-order planning," in *Proceedings of the Second International Conference on Artificial Intelligence Planning Systems*, 1994, pp. 188–193.

[30] B. Babenko, M.-H. Yang, and S. Belongie, "Visual tracking with online multiple instance learning," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2009.

[31] Y. HaCohen, N. Chiprut, B. Brazowski, D. Shalem, D. Moshe, E. Richardson, E. Levin, G. Shiran, N. Zabari, O. Gordon, P. Panet, S. Weissbuch, V. Kulikov, Y. Bitterman, Z. Melumian, and O. Bibi, "LTX-Video: Realtime video latent diffusion," *arXiv preprint arXiv:2501.00103*, 2024.

[32] Lightricks, "LTX-Video: Official model card and checkpoints," n.d., 13B 0.9.8 distilled checkpoint; accessed September 24, 2026. [Online]. Available: https://huggingface.co/Lightricks/LTX-Video
[33] T. Liu and W. S. Lee, "Can vision-language models solve the shell game?" *arXiv preprint arXiv:2603.08436*, 2026.

APPENDIX A
MEMORY RECORDS AND UPDATE SEMANTICS

A. Stored Records and References

Accepted observations become immutable events containing entity identifiers, predicate changes, confidence values, and frame references. Events create revisable facts, while subgoal records retain completion and recovery evidence. The `supersedes_fact_ids` and `depends_on_fact_ids` fields trace each update from its visual evidence to the affected goal and repair.

B. Boundary Updates and Recovery

A visual relation is accepted when identities are grounded, confidence reaches 0.8, and two distinct observations agree; missing detections alone do not create negative facts. At stationary boundaries, accepted events supersede conflicting records, invalidate unsupported descendants, and expose affected task obligations. The controller then compiles or updates a deduplicated repair using the currently supported destination. A completed condition losing support does not authorize an arbitrary new plan: the repair remains bounded by the affected skill contract.

Figure 7 joins evidence, supersession, and recovery references from an assisted partial T1b rollout using the recompute-on-demand control. It illustrates how those references can be inspected, not a successful explicit-invalidation ablation. The raw history and the interpretation of that history must remain distinguishable when inspecting any rollout.

C. A Worked Support-Withdrawal Example

Consider an accepted containment record r_s and a separately accepted bowl-location record r_l. Their conjunction supports an inferred object location r_d, and a verified placement supports completion r_g. If a new event supersedes r_s, the explicit strategy traverses its outgoing support links and withdraws active descendants. The ledger still contains the placement event and the later intervention; r_l remains active because it did not depend on the withdrawn containment. The object location and completion must now be established again before the transport goal can be treated as satisfied.

This example also explains the difference between negative and unsupported state. An accepted observation that the object is outside the bowl can create an active negative containment fact. The previous derived destination is merely invalidated: loss of containment does not by itself tell the controller where the object landed. Recovery therefore needs current grounding rather than interpreting every invalidated proposition as its opposite.

TABLE III
PHYSICAL OUTCOME COUNTS. ENTRIES ARE FULL / PARTIAL / FAILURE; THEIR SUM IS THE METHOD'S TASK DENOMINATOR. ASSISTED RUNS REMAIN INCLUDED.

Task	Vanilla G0.5	HiMe adaptation	REMM
T1a	0 / 4 / 1	2 / 3 / 0	2 / 2 / 0
T1b	0 / 5 / 0	0 / 2 / 1	3 / 2 / 0
T2	0 / 2 / 3	0 / 5 / 0	1 / 3 / 0
T3	0 / 1 / 2	0 / 0 / 3	4 / 0 / 0
Total	0 / 12 / 6	2 / 10 / 4	10 / 7 / 0

APPENDIX B
PHYSICAL EVALUATION AND ACCOUNTING

A. Shared Executor and Baseline Boundary

The physical comparison uses Galaxea R1 Lite with shared cameras and a frozen G0.5 executor. All methods dispatch language instructions through the same motor-policy interface. Our HiMe adaptation preserves planner and sentry roles, episodic and procedural stores, and memory addition, update, and deletion [10]. Its configured planner and sentry use Qwen3-VL-4B-Instruct, MiniLM-L6-v2 retrieval, and an eight-frame working context, with prompts and instruction parsing adapted to G0.5. These differences from the original model stack are why we refer to it as an adaptation.

Editable memory is not unique to REMM. The more specific distinction is that REMM retains the supporting records for a completed condition and connects their withdrawal to a repair obligation. Comparisons between complete systems also include perception, verification, instruction selection, and stopping behavior. They do not isolate graph propagation as the only changed component.

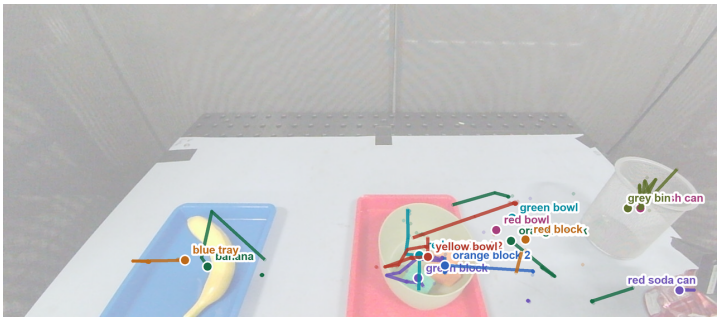
B. Assistance and Outcome Labels

Table III reproduces the task-separated outcome ledger. Assistance is recorded separately from outcome; the export does not relabel a completed run as autonomous simply because it reached the goal. The assistance fields contain 12 recorded-none, 0 task-assisted, and 6 unrecorded vanilla trials; 6, 2, and 8 HiMe trials; and 10, 4, and 3 REMM trials, respectively. Unrecorded assistance does not mean unassisted.

The two task-assisted REMM full completions are a T1b run with an orange-block nudge and a T2 run with help completing strawberry pickup. We retain the recorded outcome labels while making this assistance visible. Controller stop, return-home motion, and a policy's completion claim are not independent evidence of physical success. The saved ledger and operator annotations support the plotted export; a complete independent re-adjudication is not established by these figures.

C. Reading the Radial Profile

The radial plot groups five measures within each task sector. Progress uses the recorded checkpoint fraction, with the stated fallbacks for missing progress. Decision accuracy uses available high-level-choice labels: recovery timing or




```
{ "event_id": "..._56_orange_block_01",
 "evidence_refs": [
  "000056_head",
  "000056_left_wrist",
  "000056_right_wrist"],
 "fact_id": "..._56_orange_block_01_fact_01",
 "predicate": "inside", "value": false,
 "status": "active",
 "supersedes_fact_ids": ["..._fact_02"],
 "recovery_history": {
  "event_id": "..._56_orange_block_01" },
 "decision": "recover_affected_object" }
```


Fig. 7. **Recorded T1b memory example.** The trace contains 2,408 head-camera positions for 15 identities; gaps are disconnected tracks. The joined excerpt exposes evidence-to-event, fact-supersession, and recovery references. This assisted partial rollout uses the recompute-on-demand control and is included only for mechanism inspection.

destination for T1, complete order for T2, and target-cover choice for T3. Non-failure completion includes both full and partial outcomes. A run can therefore have a correct decision or be non-failed without completing the task.

The labeled decision denominators are 10/8/9 for vanilla/HiMe/REMM in pooled T1, 3/3/4 in T2, and 3/3/4 in T3. Their correct-decision counts are 3/5/9, 0/0/1, and 1/0/4, respectively. Each metric excludes its own blank labels, so these denominators need not match the outcome totals.

Time and query efficiency are relative median-cost scores, not elapsed seconds or query counts. Each task's lowest observed median receives 100; a method with twice that median would receive 50. T2 time and query medians use two of five baseline trials and all four REMM trials. This available-record comparison is useful for inspecting overhead, but not for concluding that a system is cheaper at the same level of task completion. Supervisory VLM calls are excluded from the G0.5-query axis, and the plotted export does not establish total model-inference cost or complete memory-traffic accounting.

Appendix C
Video Baseline Interfaces and Scoring

A. What Each Baseline Predicts

The offline methods produce a high-level readout, not action chunks. For T2, the scored output contains a predicted cue sequence, a first-object grounding, a destination, and a first language command. For T3, the output selects a final left, middle, or right cover position with its lift command. These are the fields consumed by the scorer; formatting a language subtask for a downstream executor does not demonstrate that the executor would carry it out.

The no-memory VLM reads the final frame without a retained demonstration. HiMe adds retrieved episodic and procedural memory. The MemER-style adaptation exposes retained visual keyframes to a high-level planner; the RoboMME-style adaptation exposes symbolic subgoal memory. Both use frozen Qwen in this comparison rather than the original trained model stack. The terms MemER-style and RoboMME-style identify those memory-interface choices, not official reproductions of MemER or RoboMME's policy variants [1], [9].

The resulting language subtasks are compatible with a downstream $\pi_{0.5}$ interface [6], but no $\pi_{0.5}$ motor inference or physical rollout occurs. REMM's T2 path instead joins sampled visual evidence to event memory and a task-specific grounding compiler; its T3 path follows the initially bound target cover using MIL tracking [30]. Observation selection and compute differ across these paths. The matched clips control the diagnostic input set, not every aspect of the inference budget.

B. Prediction Versus Expected Answer

For each T2 clip, the scorer retains the demonstrated three-cue sequence and the expected first object, destination, and command. Cue LCS is the length of the longest common subsequence between predicted and demonstrated cues. Exact sequence requires the entire three-item order to match. First-object correctness tests the selected current object corresponding to the first remembered cue; destination correctness tests the required receiving object or region. Command correctness checks action, object or color, and destination together using the fixed noun-equivalence rules. A correct sequence does not automatically receive first-object or command credit.

Across 12 clips, three cue positions per clip give a maximum LCS total of 36. Each other T2 component is binary per clip and therefore has a denominator of 12. For example, the MemER-style row's 36 LCS and 12 exact sequences indicate complete sequence recall on this set. Its zero first-object and command counts are separate scoring failures, not contradictions of those recall scores. Reporting both makes the memory-to-action gap visible without altering any result.

For T3, the expected answer is the final position of the cover bound to the target, not its original position before the shuffle. We report one hidden-target score because the target selection and corresponding lift command represent the same decision in this diagnostic. The scorer's answer and generation metadata remain withheld from the methods; the reported results cover

only the 12 T2 and 30 T3 clips in this snapshot.

C. Partial Accuracy and Full T2

Let a_m^{LCS} be method m's cue LCS total and let $a_m^{\mathrm{seq}}, a_m^{\mathrm{obj}}, a_m^{\mathrm{dest}}, a_m^{\mathrm{cmd}}$ be its four binary-field totals. The plotted composite is

$$A_m^{\mathrm{partial}} = \frac{100}{5}\left(\frac{a_m^{\mathrm{LCS}}}{36} + \frac{a_m^{\mathrm{seq}}}{12} + \frac{a_m^{\mathrm{obj}}}{12} + \frac{a_m^{\mathrm{dest}}}{12} + \frac{a_m^{\mathrm{cmd}}}{12}\right). \tag{11}$$

Each colored segment in Fig. 5 contributes one fifth of its normalized metric accuracy. Normalization is important: summing the raw counts would give cue LCS three times the available points of any other component. The chosen average gives the five displayed metrics equal weight, while still giving recall two related components. It is a descriptive summary, not five independent trials or a probability of motor success.

Full T2 instead counts clips on which exact sequence, first object, destination, and command are all correct together. It cannot be reconstructed by averaging those marginal totals: a method can get different fields right on different clips. REMM's 8/12 full decisions and RoboMME-style's 5/12 therefore describe a stricter outcome than their respective 86.1% and 73.9% partial scores. The full and T3 columns remain separate from physical-trial success.

Appendix D
Scope of the Mechanism Evidence

The direct-update, recompute-on-demand, and explicit-invalidation strategies distinguish three ways of responding to the same accepted change. Direct update revises the physical record without reopening its dependent goal. Recompute-on-demand checks saved postconditions when deciding what to do next. Explicit invalidation follows stored support links and exposes unsupported goals as soon as their required parents become inactive. This distinction concerns state-update logic, not which strategy can physically grasp an object.

The worked example and recorded references make the revision path inspectable, but the reported physical outcomes compare complete deployed systems. Establishing a separate benefit of support-graph propagation over recomputation would require fixed perception, accepted evidence, goal checks, and execution, followed by matched rollout comparisons. Likewise, nominal T2 and deterministic T3 primarily test retention and grounding. We do not treat their scores as evidence that dependency propagation alone caused the observed gains.